

Segment Tree learning notes

Round 4

目标是把「区间查询 range query」和「单点更新 update」都从 $O(n)$ 降到 $O(\log n)$ 。

典型：区间和 / 最大值 / 最小值。

1. 为什么需要Segment Tree 线段树？

数组 $V=[2,7,1,5,3,4]$ ，查询 $\text{sum}(2:5)=1+5+3+4$ 。朴素做法每次扫一遍：for $i=l..r$: $\text{ans} += V[i]$ ，单次 $O(n)$ ；查询次数多 + 还会更新时就不可用。

2. Prefix Sum 为什么不够？

前缀和 $C[i]=V[0]+\dots+V[i]$ ，查询 $\text{sum}(l:r)=C[r]-C[l-1]$ 是 $O(1)$ 。但一旦 $V[k]$ 改了，后面所有前缀都要改，更新 $O(n)$ 。

数据结构	查询	更新
Prefix Sum	$O(1)$	$O(n)$
Segment Tree	$O(\log n)$	$O(\log n)$

3. Segment Tree 是什么？

不断把区间二分： $0..15 \rightarrow (0..7, 8..15) \rightarrow \dots \rightarrow$ 直到单点。形成一棵完全二叉树：叶子存 $V[i]$ ，内部节点存某个区间的信息（例如求和： $\text{node} = \text{左儿子和} + \text{右儿子和}$ ）。

4. 节点表示什么？

每个节点对应一个区间： $(0:1)=V[0]+V[1]$ ， $(0:3)=V[0]+\dots+V[3]$ ，根 $(0:15)$ 表示整个数组。

5. 为什么查询是 $O(\log n)$ ？

查询 $\text{sum}(3:10)$ 不用逐个加，而是用少量区间节点覆盖： $V[3] + \text{sum}(4:7) + \text{sum}(8:9) + V[10]$ （这就是“区间分块”）。树高是 $\log n$ ，每层最多处理常数个节点，因此 $O(\log n)$ 。

6. L/R 指针规则

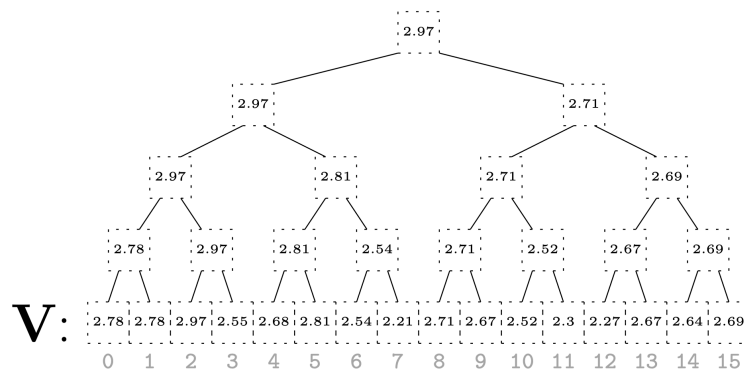
查询时 L 指向左端叶子、R 指向右端叶子，然后一起向上跳：

- 直观理解：父区间可能包含“多余部分”，只有上述两种“刚好贴边”的情况才能整块拿。



8. 关键条件：为什么不只支持 sum？

线段树要求区间信息可合并： $f(a:b)$ 可以由 $f(a:c)$ 和 $f(c+1:b)$ 快速得到，所以可以做 $\text{sum} / \text{max} / \text{min} / \text{gcd} / \text{乘积}$ 等。



The figure shows a segment tree for interval maximum queries.

9. Array-based Segment Tree

不用真建树，用数组像 heap 一样存：

- 节点 i 的左儿子 $2i$ ，右儿子 $2i+1$ ，父节点 $i/2$
- off = 不小于 n 的最小 2 的幂；叶子放在 $S[off+i]$ ，再自底向上 build

1-indexed 写法 根在 $S[1]$ ，0-indexed 写法 根在 $S[0]$

